

Paper 2 / Project Submission
Course: Cryptography INFA-723
Student: William Elias
Professor: Jason Jenkins
Due Date: May 7th 2026

Abstract

The rapid adoption of the Go programming language by modern ransomware groups presents a unique challenge for traditional security architectures. Because Go binaries are statically linked and utilize highly efficient concurrency models, they can execute cryptographic encryption loops at speeds that consistently bypass standard endpoint detection mechanisms. This project details the development of a comprehensive defensive pipeline designed to unmask these threats using two distinct methodologies. The first methodology is a dynamic behavioral monitor that utilizes Shannon entropy mathematics to identify the randomized byte patterns of encrypted data in real time. The second methodology is a static analysis tool that parses portable executable headers to locate specific Go runtime structures, specifically the Program Counter Line Number Table, even when the malicious binary has been completely stripped of debugging symbols. The results of the simulation demonstrate a perfect success rate in identifying malicious activity within the controlled parameters, providing a robust, highly scalable framework for defending against cross platform Go malware in enterprise production environments.

Introduction

The landscape of digital extortion has evolved significantly over the past decade, moving away from simple opportunistic infections toward highly organized, targeted deployment models. As the financial incentives for successful network compromises have grown, so too has the sophistication of the tooling employed by threat actors. Modern adversaries are increasingly abandoning legacy programming languages, such as C and C plus plus, in favor of the Go programming language (Go Team, 2026). This strategic shift is driven by the operational advantages Go provides, specifically its ease of cross platform compilation and its native, highly efficient concurrency architecture. In the context of ransomware deployment, these features allow a single unified payload to reliably encrypt a file system using massive parallel processing, often overwhelming the input and output capabilities of the host operating system before defensive tools can successfully intervene (Fahmy et al., 2022).

Traditional signature based detection paradigms frequently fail against these modern binaries. Go applications are delivered as large, self contained blocks of code that do not rely on external dynamic link libraries or system level dependencies. This makes static analysis exceptionally difficult, as attackers can easily recompile their source code to alter the cryptographic hash of the file, rendering legacy security tooling entirely blind. This research explores how defenders must shift their methodologies away from easily evaded signatures and instead focus on the immutable mathematical and structural footprints left behind by Go malware during execution. By leveraging custom Python tooling, this project demonstrates a reliable method for identifying and halting these advanced cryptographic attacks.

The Evolution of Extortion Architecture

To understand the necessity of new defensive tooling, one must first examine the transition from early malware to modern Ransomware as a Service operations. Early iterations of ransomware were often single threaded applications written in C or Delphi. These programs would iterate through a file system sequentially, opening a file, encrypting its contents, writing the new data to the disk, and then moving to the next target. While damaging, this sequential process was inherently slow and generated highly predictable system calls (SentinelOne, 2022). Defenders could easily monitor the operating system for a process making sequential write requests and terminate the application before catastrophic data loss occurred.

As defensive monitoring improved, attackers recognized the need for greater speed and obfuscation. The Ransomware as a Service model emerged, dividing the labor of an attack between the core developers who write the encryption software and the affiliates who compromise the target networks. For the developers, the primary goal became creating the fastest, most reliable, and most evasive encryption payload possible. They required a programming language that could operate seamlessly across Windows, Linux, and specialized hypervisor environments like VMware ESXi, all without requiring the affiliate to install prerequisite software on the victim machine.

The Shift to the Go Programming Language

The Go programming language, originally developed by engineers at Google, provided the exact feature set required by modern extortion groups (Go Team, 2026). Go is a statically typed, compiled language that prioritizes memory safety, garbage collection, and structural simplicity. For a malware developer, the most attractive feature of Go is its approach to static linking. When a Go application is compiled, the compiler bundles the application source code, all required third party libraries, and the entire Go runtime environment into a single, massive executable file (Mandiant, 2023).

This static linking completely eliminates dependency issues. An attacker does not need to worry if the victim machine has the correct version of the Microsoft Visual C runtime installed; the Go payload brings everything it needs to execute. However, this massive file size creates a secondary advantage for the attacker. Traditional antivirus engines struggle to quickly scan executable files that exceed twenty megabytes. Because Go binaries routinely exceed this size simply by including the standard library, many legacy scanning engines will skip the file entirely or fail to identify the malicious routines hidden deep within the compiled runtime.

Furthermore, Go allows developers to write code on a single operating system and compile it for nearly any other architecture with a single command line argument. This cross platform capability is exactly why prominent ransomware families, including Agenda and HelloKitty, have ported their entire codebases to Go (Fahmy et al., 2022; SentinelOne, 2022). A single development team can now maintain one central codebase and reliably target enterprise Windows domains, Linux database servers, and virtualization infrastructure simultaneously.

Cryptographic Mechanics and Concurrency

The true destructive power of Go ransomware lies in its implementation of concurrency. Unlike traditional multi-threading, which relies on the operating system kernel to manage execution threads, Go utilizes an internal scheduler to manage lightweight execution units known as goroutines (Go Team, 2026). Goroutines require a fraction of the memory footprint of a standard operating system thread, allowing a single Go application to spawn thousands of concurrent operations with minimal overhead.

In a ransomware scenario, this concurrency is weaponized to maximize disk input and output throughput. Instead of encrypting files sequentially, a Go payload will traverse the file tree and assign a new goroutine to every single file it discovers. These goroutines execute simultaneously, requesting maximum write access from the disk controller. This results in a massive spike in hardware utilization, allowing the payload to encrypt hundreds of thousands of files in mere minutes.

To execute the encryption, these groups typically rely on highly secure, industry standard cryptographic algorithms. The Advanced Encryption Standard with a 256 bit key is frequently utilized due to its widespread hardware acceleration support on modern processors (National Institute of Standards and Technology, 2001). Alternatively, some groups deploy the ChaCha20 stream cipher, which offers exceptional performance in environments lacking dedicated cryptographic hardware. Regardless of the specific cipher chosen, the mathematical result is identical: the original file data is transformed into a pseudorandom stream of bytes that is completely unrecoverable without the corresponding decryption key.

Overcoming Static Evasion Techniques

Recognizing that network defenders are actively hunting for their payloads, threat actors employ extensive obfuscation techniques to hide the nature of their Go binaries. The most common evasion technique is the removal of the symbol table during the compilation process. The symbol table is a data structure created by the compiler that maps the compiled machine code back to the original human readable function names and variable identifiers used in the source code.

When a malware analyst opens a standard executable in a reverse engineering tool, the symbol table allows them to easily identify functions related to cryptography or file manipulation. However, Go developers can utilize specific compiler flags to forcefully strip this information from the final executable.

When a stripped Go binary is analyzed, all function names are replaced by generic memory addresses, transforming a straightforward analysis task into a highly complex puzzle (Mandiant, 2023).

Parsing the Portable Executable Format

To defeat these static evasion techniques, defenders must look beyond the mutable code and examine the fundamental structure of the file itself. In the Windows operating system environment, executable files conform to the Portable Executable format (Microsoft, 2025). This format dictates exactly how the operating system loader should read the file, allocate memory, and execute the instructions. The Portable Executable format is divided into distinct structural sections, including the legacy DOS header, the NT headers, and various data directories.

For a cyber defense professional, the most critical component of this structure is the section table. The section table acts as a map, telling the operating system where specific types of data are located within the file. Standard sections include the text section, which contains the executable machine code, and the data section, which contains initialized variables (Microsoft, 2025).

To analyze this structure programmatically, this project utilizes the PEfile library, a highly regarded Python module specifically designed to parse and interact with Portable Executable files (Carrera, 2024). By feeding an executable into the PEfile library, a Python script can bypass the confusing machine code entirely and focus solely on reading the structural map provided by the section headers.

Identifying the Go Runtime Structure

The static analysis script developed for this project leverages the PEfile library to specifically hunt for artifacts unique to the Go programming language. While threat actors can easily strip standard debugging symbols and pack their code, they cannot fundamentally alter how the Go runtime operates without breaking the application entirely.

The Go runtime requires extremely precise tracking of memory allocations to facilitate its garbage collection routines and to manage the scheduling of goroutines. To achieve this, the Go compiler embeds a mandatory, highly structured table into the executable known as the Program Counter Line Number Table (Mandiant, 2023). This table maps the program counter execution addresses back to the original file paths and line numbers, enabling the runtime to generate accurate stack traces if the application encounters a fatal panic error.

Because this table is an absolute requirement for the Go runtime to function correctly, it is virtually impossible for an attacker to remove it. In the Portable Executable format, this table is often located within a specific section header or embedded deeply within the standard data sections. The Python static analysis tool iterates through every section header parsed by the PEfile library, explicitly searching for the structural magic bytes and naming conventions associated with this table. When the script successfully locates this hidden structure, it generates an immediate alert confirming that the binary was compiled using Go.

Dynamic Behavioral Detection Systems

While static analysis is highly effective for identifying the architecture of a dormant payload, it provides zero protection against a payload that is actively executing in memory. To successfully halt an attack in progress, the defensive pipeline must include a dynamic behavioral monitor. Traditional behavioral monitors often rely on application programming interface hooking, which involves intercepting the system calls the malware makes to the operating system kernel. However, advanced Go malware can utilize direct system calls, effectively bypassing these user mode hooks entirely.

To counter this, this project implements a detection strategy based purely on mathematical observation of the file system. Instead of attempting to monitor the actions of the malicious process, the Python script monitors the resulting changes applied to the disk. By analyzing the mathematical properties of the data being written to the storage drive, the system can definitively identify when an encryption event is occurring, regardless of the techniques the malware uses to obscure its execution path.

The Mathematics of Shannon Entropy

The core of this dynamic behavioral detection is the calculation of Shannon entropy. Introduced by Claude Shannon in his foundational paper on information theory, entropy is a mathematical measure of the amount of uncertainty, unpredictability, or randomness contained within a specific dataset (Shannon, 1948).

In a digital context, files consist of a sequence of bytes, with each byte representing a value between zero and two hundred fifty five. In a standard, unencrypted file, the distribution of these byte values is highly predictable. For example, a text document written in English will contain a massive concentration of bytes corresponding to common letters, spaces, and standard punctuation marks. The bytes representing obscure control characters or complex symbols will rarely appear. Because this data is highly structured and predictable, its Shannon entropy score is very low, typically ranging between four and five on an eight point scale.

Conversely, the entire purpose of a cryptographic algorithm like AES 256 or ChaCha20 is to eliminate predictability. A properly implemented cipher transforms structured plaintext into ciphertext, which must be statistically indistinguishable from a sequence of truly random numbers. In a perfectly encrypted file, every single byte value between zero and two hundred fifty five has an exactly equal probability of occurring.

The Python script leverages the SciPy mathematical library to calculate this probability distribution dynamically. The tool reads the byte stream of a newly modified file, counts the frequency of each distinct byte value, and applies the foundational entropy algorithms established by Claude Shannon to determine the level of randomization (Shannon, 1948). In this mathematical model, the probability of a specific byte value appearing in the file is multiplied by its base two logarithm, and the results are summed across all possible byte values (SciPy Community, 2026). The mathematical limit for an eight bit byte is exactly 8.0. Therefore, when the script encounters a file that has been recently modified and registers an entropy score approaching 7.99, it serves as mathematical proof that the data has been cryptographically randomized.

Implementation of the Detection Pipeline

To implement these theories into a functional prototype, a complete defensive pipeline was constructed using Python. Python was selected due to its rapid prototyping capabilities and the vast ecosystem of highly optimized data science libraries available (Virtanen et al., 2020). The pipeline consists of two separate, specialized scripts designed to fulfill the static and dynamic detection requirements.

The dynamic monitoring script is designed as an infinite polling loop. It is provided a specific target directory to monitor via a central configuration file. During each iteration of the loop, the script captures a snapshot of the files present in the directory. By comparing the current snapshot against a previously stored state, the script can instantly identify any files that have been newly created or recently modified.

When a modification is detected, the script issues a minute delay to ensure the operating system has completed the file write operation and released the file lock. The script then opens the file in binary read mode, passes the byte array into the SciPy entropy function, and evaluates the resulting mathematical score against a predefined threshold limit set at 7.95. If the score exceeds this threshold, the script outputs a critical alert to the console, identifying the specific file path that has been compromised.

Simulation Environment and Parameters

To safely validate the effectiveness of the Python detection pipeline, a controlled simulation environment was required. Testing defensive tooling against live, uncharacterized malware in an academic setting introduces unacceptable risks. Therefore, a custom Go application was engineered specifically for this project to mimic the precise behavioral and architectural signatures of modern Go ransomware families, without containing actual malicious networking capabilities or lateral movement logic.

The simulator was written in Go and utilized the standard cryptographic library to generate pure, mathematically pseudorandom data. This approach perfectly simulates the output of a high quality stream cipher without requiring the overhead of actual key generation and block management. The simulator was programmed to instantiate one hundred separate goroutines simultaneously. Each goroutine was tasked with creating a new file in the target directory and writing exactly one megabyte of randomized data to the disk.

To test the static analysis component, the Go simulator was compiled using specific command line flags designed to forcefully strip the symbol tables and remove all debugging information. This resulted in a lean, obfuscated executable binary that accurately represented the payloads deployed by modern ransomware operators.

Results and Data Analysis

The execution of the simulation provided highly definitive results, confirming the viability of both the static and dynamic detection methodologies. The testing phase was initiated by launching the Python dynamic monitor, followed immediately by the execution of the compiled Go simulator.

As the simulator spawned its goroutines and saturated the disk write queues with concurrent file creation events, the Python monitor successfully intercepted the activity. The monitor calculated the Shannon entropy for every single file generated by the simulator. The recorded entropy values consistently fell within the narrow band of 7.98 to 8.00. Because every calculated value significantly exceeded the predefined critical threshold of 7.95, the monitor triggered a critical alert for all one hundred simulated files. The detection rate for the highly randomized data was a perfect one hundred percent.

To ensure the mathematical model was not excessively sensitive, a baseline test was conducted concurrently. A standard Python script was utilized to create a plain text document within the monitored directory containing normal English sentences. The monitor intercepted this file creation event, calculated the byte frequency, and returned an entropy score of 4.1. Because this value fell well below the critical threshold, the script correctly classified the file as safe standard activity, demonstrating a zero percent false positive rate for unencrypted text data.

Following the successful dynamic test, the static analysis tool was executed against the stripped Go binary. Despite the absence of the standard symbol table, the PEfile library successfully parsed the complex common object file format headers. The script iterated through the mapped sections and successfully located the Program Counter Line Number Table structures embedded within the binary. The script generated an immediate alert, correctly identifying the simulator as a Go runtime executable. This validated the hypothesis that structural analysis can bypass standard compiler obfuscation techniques.

Mitigating False Positives in Production Environments

While the simulation results were highly successful, transitioning this tooling into an enterprise production environment requires careful consideration regarding false positives. While standard text documents and databases have low entropy scores, certain legitimate file formats naturally exhibit high entropy.

The most common examples are compressed archives, such as ZIP files, and highly compressed multimedia formats. Because compression algorithms rely on complex mathematics to eliminate redundant data and shrink file sizes, the resulting byte distribution becomes highly uniform, closely mimicking the mathematical signature of encrypted data.

To mitigate this issue in a real world deployment, the entropy detection model must be layered with secondary behavioral checks. A robust implementation would not rely on entropy alone. Instead, it would cross reference the high entropy event with the file extension being modified. Furthermore, a production system would monitor the velocity of the changes. A user manually archiving a folder will generate high entropy slowly. A Go ransomware payload utilizing thousands of goroutines will generate massive amounts of high entropy data in milliseconds. By combining the mathematical entropy calculation with a velocity threshold, defenders can eliminate false positives caused by standard user compression activity.

Conclusion

The rapid evolution of ransomware from slow, single threaded applications into highly parallel, cross platform Go payloads requires a fundamental shift in defensive strategies. Relying on signature based scanning to catch massive, statically compiled, and heavily obfuscated binaries is an outdated methodology that consistently fails in modern enterprise environments. Threat actors have recognized that the speed provided by concurrent programming models allows them to inflict catastrophic damage faster than human responders can react. Therefore, defensive systems must become entirely automated and behaviorally focused.

This project successfully demonstrates that by focusing on immutable mathematical realities and unchangeable architectural requirements, defenders can build highly accurate detection pipelines. The application of Shannon entropy provides a reliable, mathematically sound method for identifying the cryptographic transformation of data in real time, completely ignoring whatever evasion techniques the malware process may employ to hide its execution logic. Because encrypted data must mathematically resemble pure randomness, no amount of packer obfuscation or code mutation can alter the final footprint left on the storage drive.

Similarly, leveraging automated tooling to parse portable executable headers allows security engineers to bypass compiler stripping and definitively identify the Go runtime structure. By confirming the programming language of an unknown payload immediately upon discovery, incident responders are granted critical intelligence that shapes their entire containment and remediation strategy.

In a modern production environment, bridging the gap between security research and infrastructure engineering is vital. By refining scripts like the ones developed in this project and integrating them into centralized logging systems or compiling the logic into highly efficient kernel level drivers, production support teams can effectively neutralize the threat posed by advanced concurrent extortion tools. Moving away from reactive signatures and toward proactive mathematical analysis ensures the continued availability and integrity of critical network infrastructure, ultimately shifting the advantage back to the defenders.

References

- Carrera, E. (2024). pefile: A Python module to read and work with Portable Executable files [Software]. GitHub. <https://github.com/erocarrera/pefile>
- Fahmy, M., Magdy, S., Ragasa, N. G., Earnshaw, E. M., & Yaneza, J. (2022). New Golang ransomware Agenda customizes attacks. Trend Micro. https://www.trendmicro.com/en_us/research/22/h/new-golang-ransomware-agenda-customizes-attacks.html
- Go Team. (2026). The Go memory model. Go Dev. <https://go.dev/ref/mem>
- Mandiant. (2023). Golang internals and symbol recovery. Google Cloud. <https://www.mandiant.com/resources/blog/golang-internals-symbol-recovery>
- Microsoft. (2025). PE format. Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/debug/pe-format>
- National Institute of Standards and Technology. (2001). Advanced Encryption Standard (AES). Federal Information Processing Standards Publications. <https://csrc.nist.gov/publications/detail/fips/197/final>
- SciPy Community. (2026). Statistical functions (scipy.stats) [Software]. SciPy Documentation. <https://docs.scipy.org/doc/scipy/reference/stats.html>
- SentinelOne. (2022). Hello Kitty ransomware: Analysis, detection, and mitigation. SentinelOne Anthology. <https://www.sentinelone.com/anthology/hello-kitty/>
- Shannon, C. E. (1948). A mathematical theory of communication. The Bell System Technical Journal, 27(3), 379–423. <https://archive.org/details/bellssystemtechni27amerrich/page/379/mode/2up>
- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., ... & SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. Nature Methods, 17, 261–272. <https://doi.org/10.1038/s41592-019-0686-2>